UCS420 Lab Assignment 9 — NLP Text Preprocessing

Enter your roll number and name, then run all cells.

```
ROLL_NO = 1024170368          # <-- your full roll number
NAME = "Rahul"            # <-- your name

import nltk
for pkg in ["punkt", "punkt_tab", "stopwords", "wordnet", "omw-1.4",
            "averaged_perceptron_tagger", "averaged_perceptron_tagger_eng"]:
    nltk.download(pkg, quiet=True)
assert ROLL_NO > 0 and NAME, "Enter your roll number and name first"
```

Dataset generator — do not modify

```
import random, hashlib
import pandas as pd

def generate_dataset(roll_no):
    """Return (DataFrame of tickets, focus ticket IDs, fingerprint) for a roll number. Do not modify."""
    rng = random.Random(int(roll_no))

    subjects = [("LMS", ["LMS", "lms"]), ("Wi-Fi", ["Wi-Fi", "WiFi", "wifi"]), ("VPN", ["VPN"]),
                ("email", ["email", "e-mail"]), ("password", ["password"]), ("portal", ["student portal"]),
                ("projector", ["projector"]), ("laptop", ["laptop"]), ("MATLAB", ["MATLAB", "Matlab"]),
                ("Python", ["Python"]), ("printer", ["printer"]), ("server", ["server"])]
    actions = ["is not working", "keeps disconnecting", "failed to open", "is running slowly",
               "crashes while opening", "stopped responding", "is showing an error", "was working yesterday",
               "has stopped syncing", "keeps asking for login"]
    user_actions = ["cannot access", "can't open", "couldn't connect to", "am unable to use",
                    "tried restarting", "was locked out of", "keep getting logged out of", "reinstalled"]
    contexts = ["after resetting my password", "during the lab", "on my laptop", "since yesterday",
                "after the latest update", "when I use mobile hotspot", "before the assignment deadline",
                "while uploading the file", "after restarting the system", "when connected to campus Wi-Fi"]
    endings = ["Please help.", "Can you check this?", "I have tried several times!", "It worked yesterday.",
               "I don't know what changed.", "The same problem occurs again.", "Is there another solution?",
               "It works on another computer.", "This is urgent.", "Please resolve it."]
    challenges = {
        "negation": ["The Wi-Fi works, but the VPN doesn't.", "Why isn't the portal opening?",
                     "I can't login and I didn't change anything.", "It's not the laptop; it's the server.",
                     "Nothing works and nobody has replied.", "The printer won't print; it isn't jammed."],
        "numbers": ["Python 3.12 was installed yesterday.", "The file is 95.5% uploaded.",
                    "Error code 0x80070005 appears.", "I need Windows 11 by 5.30 p.m. today.",
                    "Only 2 of 30 PCs boot after 3.5 hours.", "Version 2.0.1 crashes at 99%."],
        "identifiers": ["Email me at helpdesk@university.edu.", "See https://lms.thapar.edu/help for details.",
                        "My roll no. is 1023-03-021.", "Ticket #4521 is still open.",
                        "Call me on +91 98765 43210 or 0175-239-3201.",
                        "Write to it.helpdesk@thapar.edu or visit www.thapar.edu/it."],
        "punctuation": ["I tried again... still no response!!!", "The LMS says 'Not Submitted'.",
                        "Dr. Rao's lab PC (No. 14) is down.", "Is it fixed?? Pls reply ASAP :(",
                        "The state-of-the-art lab (Lab-3) is closed.", "Re-installing didn't help; log-in still fails!"],
    }

    tickets = []
    for _ in range(8 + int(roll_no) % 5):
        _, variants = rng.choice(subjects)
        s = rng.choice(variants)
        template = rng.randrange(3)
        if template == 0:
            text = f"{s} {rng.choice(actions)} {rng.choice(contexts)}. {rng.choice(endings)}"
        elif template == 1:
            text = f"I {rng.choice(user_actions)} the {s} {rng.choice(contexts)}. {rng.choice(endings)}"
        else:
            text = f"The {s} {rng.choice(actions)}. {rng.choice(endings)}"
        tickets.append(text[0].upper() + text[1:])
    for category in ["negation", "numbers", "identifiers", "punctuation"]:
        tickets.append(rng.choice(challenges[category]))
    rng.shuffle(tickets)

    df = pd.DataFrame({"Ticket_ID": [f"T{i + 1:02d}" for i in range(len(tickets))], "Ticket_Text": tickets})
    focus = [f"T{i:02d}" for i in sorted(rng.sample(range(1, len(tickets) + 1), 2))]
    fingerprint = hashlib.md5(("|".join(tickets) + str(int(roll_no))).encode()).hexdigest()[:8].upper()
    return df, focus, fingerprint
```

Preprocessing pipeline — do not modify

```
import string
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords

STOP = set(stopwords.words("english"))

def is_punct(token):
    """A token is punctuation if EVERY character in it is in string.punctuation (e.g. '.', '...', '``', "'")."""
    return all(ch in string.punctuation for ch in token)

def process_ticket(text):
    """The official pipeline. Returns sentences, lowercase tokens, punctuation, stopwords and final tokens."""
    sentences = sent_tokenize(text)
    tokens = [t.lower() for s in sentences for t in word_tokenize(s)]
    punct = [t for t in tokens if is_punct(t)]
    stops = [t for t in tokens if t in STOP]
    final = [t for t in tokens if not is_punct(t) and t not in STOP]
    return {"sentences": sentences, "tokens": tokens, "punct": punct, "stops": stops, "final": final}
```

```
df, FOCUS, FINGERPRINT = generate_dataset(ROLL_NO)
print(f"Roll number: {ROLL_NO}   Name: {NAME}   Fingerprint: {FINGERPRINT}   Tickets: {len(df)}\n")
print(df.to_string(index=False))
df.to_csv(f"{ROLL_NO}_generated_dataset.csv", index=False)
```

```
Roll number: 1024170368   Name: Rahul   Fingerprint: FBAB2540   Tickets: 15

Ticket_ID                                                                  Ticket_Text
      T01    I reinstalled the MATLAB before the assignment deadline. It works on another computer.
      T02       WiFi was working yesterday while uploading the file. The same problem occurs again.
      T03                                      The Matlab stopped responding. It worked yesterday.
      T04                                           Re-installing didn't help; log-in still fails!
      T05                        Student portal is not working since yesterday. It worked yesterday.
      T06      I couldn't connect to the laptop before the assignment deadline. Can you check this?
      T07                     Lms keeps asking for login during the lab. I don't know what changed.
      T08                                      The email has stopped syncing. It worked yesterday.
      T09              I am unable to use the password since yesterday. I have tried several times!
      T10                                      The VPN keeps asking for login. It worked yesterday.
      T11                                                          Error code 0x80070005 appears.
      T12 Server keeps disconnecting before the assignment deadline. The same problem occurs again.
      T13                                                 The printer won't print; it isn't jammed.
      T14                                Write to it.helpdesk@thapar.edu or visit www.thapar.edu/it.
      T15           I cannot access the lms after resetting my password. I don't know what changed.
```

Q1. Preprocessing pipeline

Run process_ticket() on every ticket and show one table with, per ticket: sentences, tokens, punctuation tokens, stopwords, unique tokens and final tokens, plus a TOTAL row.

```
results = []

for _, row in df.iterrows():
    ticket_id = row["Ticket_ID"]
    text = row["Ticket_Text"]
    processed = process_ticket(text)

    results.append({
        "Ticket_ID": ticket_id,
        "Sentences": len(processed["sentences"]),
        "Tokens": len(processed["tokens"]),
        "Punctuation Tokens": len(processed["punct"]),
        "Stopwords": len(processed["stops"]),
        "Unique Tokens": len(set(processed["tokens"])),
        "Final Tokens": len(processed["final"])
    })

results_df = pd.DataFrame(results)

total_row = {
    "Ticket_ID": "TOTAL",
    "Sentences": "",
    "Tokens": results_df["Tokens"].sum(),
    "Punctuation Tokens": results_df["Punctuation Tokens"].sum(),
    "Stopwords": results_df["Stopwords"].sum(),
    "Unique Tokens": "",
    "Final Tokens": results_df["Final Tokens"].sum()
```

```
}

results_df = pd.concat([results_df, pd.DataFrame([total_row])], ignore_index=True)
display(results_df)

selected_ticket = df.iloc[0]["Ticket_Text"]
processed = process_ticket(selected_ticket)

print("Original ticket:")
print(selected_ticket)
print("\nActual tokens:")
print(processed["tokens"])
print("\nPunctuation tokens:")
print(processed["punct"])
print("\nStopwords:")
print(processed["stops"])
print("\nFinal tokens:")
print(processed["final"])
```

	Ticket_ID	Sentences	Tokens	Punctuation Tokens	Stopwords	Unique Tokens	Final Tokens
0	T01	2	15	2	6	13	7
1	T02	2	15	2	6	13	7
2	T03	2	9	2	2	8	5
3	T04	1	9	2	1	9	6
4	T05	2	12	2	3	10	7
5	T06	2	17	2	8	16	7
6	T07	2	16	2	6	15	8
7	T08	2	10	2	3	9	5
8	T09	2	16	2	6	15	8
9	T10	2	11	2	3	10	6
10	T11	1	5	1	0	5	4
11	T12	2	14	2	5	12	7
12	T13	1	11	2	3	10	6
13	T14	1	9	2	2	9	5
14	T15	2	18	2	9	16	7
15	TOTAL		187	29	63		95

```
Original ticket:
I reinstalled the MATLAB before the assignment deadline. It works on another computer.

Actual tokens:
['i', 'reinstalled', 'the', 'matlab', 'before', 'the', 'assignment', 'deadline', '.', 'it', 'works', 'on', 'another', 'compu

Punctuation tokens:
['.', '.']

Stopwords:
['i', 'the', 'before', 'the', 'it', 'on']

Final tokens:
['reinstalled', 'matlab', 'assignment', 'deadline', 'works', 'another', 'computer']
```

Next steps: Generate code with `results_df` New interactive sheet

Observe: Which final tokens in your data are not real words (e.g. n't, 's, ca)? Where do they come from?

Your answer: in my data, n't was a word thats not real but it was included. It comes from words like - don't. shouldn't, won't etc.

Q2. Regular expressions

On your raw tickets, use re.findall to extract: words longer than 5 letters; all numbers (keep 95.5 and 2.0.1 whole); capitalized words; words starting with a vowel. Then use re.sub to replace emails with , URLs with and phone numbers with . Your patterns must also work on TEST_LINES (given in the notebook).

```
TEST_LINES = [
    "Mail it.helpdesk@thapar.edu or a.b-c@dept.univ.ac.in; visit https://lms.thapar.edu/help or www.thapar.edu/it.",
    "Call +91 98765 43210, +91-98765-43210 or 0175-239-3201. Version 2.0.1 is 95.5% done; fee Rs 1,250.",
```

```
    "The state-of-the-art lab isn't open; log-in at 5.30 p.m. didn't work.",
]


import re

raw_tickets = df["Ticket_Text"].tolist()
all_text = raw_tickets + TEST_LINES

for text in all_text:
    print("\nOriginal text:")
    print(text)

    print("Words longer than 5 letters:")
    print(re.findall(r"\b[A-Za-z]{6,}\b", text))

    print("Numbers:")
    print(re.findall(r"\b\d+(?:\.\d+)*\b", text))

    print("Capitalized words:")
    print(re.findall(r"\b[A-Z][a-zA-Z]*\b", text))

    print("Words starting with a vowel:")
    print(re.findall(r"\b[aeiouAEIOU][a-zA-Z]*\b", text))

    print("Text after replacing emails, URLs and phone numbers:")
    text = re.sub(r"\b[\w.-]+@[\w.-]+\.[A-Za-z]{2,}\b", "<EMAIL>", text)
    text = re.sub(r"https?://\S+|www\.\S+", "<URL>", text)
    text = re.sub(r"(?<!\w)(?:\+?\d[\d -]{7,}\d)(?!\w)", "<PHONE>", text)
    print(text)
```

```
[]
```

```
[]
Capitalized words:
['Mail']
```

Observe: How many of your capitalized words are capitalized only because they start a sentence?

Your answer: Many capitalized words in my dataset are capitalized only because they appear at the beginning of a sentence. Examples include “The,” “It,” “Student,” “Server,” “Error,” “Write,” “Call,” and “Version.” However, words such as “MATLAB,” “VPN,” and “WiFi” are capitalized for other reasons, such as being product names or acronyms.

Q3. Custom tokenizer

Write my_tokenize(text) with one RegexpTokenizer pattern that keeps contractions (isn't), hyphenated words (Wi-Fi, state-of-the-art), decimals and versions (95.5, 2.0.1), emails, URLs and phone numbers as single tokens. Run it on TEST_LINES and on your tickets, and compare with word_tokenize.

```
from nltk.tokenize import RegexpTokenizer, word_tokenize

pattern = r"https?://[^\s]+?(?=[.,!?;:]?(?:\s|$))|www\.[^\s]+?(?=[.,!?;:]?(?:\s|$))|[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za

tokenizer = RegexpTokenizer(pattern)

def my_tokenize(text):
    return tokenizer.tokenize(text)

for text in TEST_LINES:
    print("\nOriginal text:")
    print(text)
    print("My tokenizer:")
    print(my_tokenize(text))
    print("NLTK tokenizer:")
    print(word_tokenize(text))

different_tickets = []

for _, row in df.iterrows():
    text = row["Ticket_Text"]
    custom_tokens = my_tokenize(text)
    nltk_tokens = word_tokenize(text)

    if custom_tokens != nltk_tokens:
        different_tickets.append(row["Ticket_ID"])

print("\nTickets tokenized differently:")
print(different_tickets)
print("Number of tickets tokenized differently:", len(different_tickets))
```

```
Original text:
Mail it.helpdesk@thapar.edu or a.b-c@dept.univ.ac.in; visit https://lms.thapar.edu/help or www.thapar.edu/it.
My tokenizer:
['Mail', 'it.helpdesk@thapar.edu', 'or', 'a.b-c@dept.univ.ac.in', ';', 'visit', 'https://lms.thapar.edu/help', 'or', 'www.th
NLTK tokenizer:
['Mail', 'it.helpdesk', '@', 'thapar.edu', 'or', 'a.b-c', '@', 'dept.univ.ac.in', ';', 'visit', 'https', ':', '//lms.thapar.

Original text:
Call +91 98765 43210, +91-98765-43210 or 0175-239-3201. Version 2.0.1 is 95.5% done; fee Rs 1,250.
My tokenizer:
['Call', '+91 98765 43210', ',', '+91-98765-43210', 'or', '0175-239-3201', '.', 'Version', '2.0.1', 'is', '95.5', '%', 'done
NLTK tokenizer:
['Call', '+91', '98765', '43210', ',', '+91-98765-43210', 'or', '0175-239-3201', '.', 'Version', '2.0.1', 'is', '95.5', '%',

Original text:
The state-of-the-art lab isn't open; log-in at 5.30 p.m. didn't work.
My tokenizer:
['The', 'state-of-the-art', 'lab', "isn't", 'open', ';', 'log-in', 'at', '5.30', 'p', '.', 'm', '.', "didn't", 'work', '.']
NLTK tokenizer:
['The', 'state-of-the-art', 'lab', 'is', "n't", 'open', ';', 'log-in', 'at', '5.30', 'p.m.', 'did', "n't", 'work', '.']

Tickets tokenized differently:
['T04', 'T06', 'T07', 'T11', 'T13', 'T14', 'T15']
Number of tickets tokenized differently: 7
```

Observe: How many of your tickets are tokenized differently by my_tokenize and word_tokenize?

Your answer: 7

Q4. Stemming and lemmatization

On your unique final tokens, compare PorterStemmer, LancasterStemmer, a RegexpStemmer with a suffix rule you choose after looking at your own words, and WordNetLemmatizer with POS tags from nltk.pos_tag. Show a table of the words where the outputs differ.

```
import nltk
from nltk.stem import PorterStemmer, LancasterStemmer, RegexpStemmer, WordNetLemmatizer
from nltk.corpus import wordnet
import pandas as pd

unique_tokens = sorted(set(
    token
    for text in df["Ticket_Text"]
    for token in process_ticket(text)["final"]
))

porter = PorterStemmer()
lancaster = LancasterStemmer()
regex_stemmer = RegexpStemmer("ing$")
lemmatizer = WordNetLemmatizer()

def get_wordnet_pos(tag):
    if tag.startswith("J"):
        return wordnet.ADJ
    if tag.startswith("V"):
        return wordnet.VERB
    if tag.startswith("N"):
        return wordnet.NOUN
    if tag.startswith("R"):
        return wordnet.ADV
    return wordnet.NOUN

tagged_tokens = nltk.pos_tag(unique_tokens)

results = []

for token, tag in tagged_tokens:
    results.append({
        "Word": token,
        "Porter": porter.stem(token),
        "Lancaster": lancaster.stem(token),
        "Regex": regex_stemmer.stem(token),
        "WordNet": lemmatizer.lemmatize(token, get_wordnet_pos(tag))
    })

comparison = pd.DataFrame(results)

different_words = comparison[
    (comparison["Porter"] != comparison["Lancaster"]) |
    (comparison["Porter"] != comparison["Regex"]) |
    (comparison["Porter"] != comparison["WordNet"]) |
    (comparison["Lancaster"] != comparison["Regex"]) |
    (comparison["Lancaster"] != comparison["WordNet"]) |
    (comparison["Regex"] != comparison["WordNet"])
]

display(different_words)
```

	Word	Porter	Lancaster	Regex	WordNet
2	another	anoth	anoth	another	another
3	appears	appear	appear	appears	appear
5	assignment	assign	assign	assignment	assignment
6	changed	chang	chang	changed	change
8	code	code	cod	code	code
9	computer	comput	comput	computer	computer
12	deadline	deadlin	deadlin	deadline	deadline
15	error	error	er	error	error
16	fails	fail	fail	fails	fail
17	file	file	fil	file	file
20	jammed	jam	jam	jammed	jam
21	keeps	keep	keep	keeps	keep
25	lms	lm	lms	lms	lm
30	occurs	occur	occ	occurs	occur
32	portal	portal	port	portal	portal
34	printer	printer	print	printer	printer
36	re-installing	re-instal	re-installing	re-install	re-installing
37	reinstalled	reinstal	reinstal	reinstalled	reinstall
38	resetting	reset	reset	resett	reset
40	server	server	serv	server	server
41	several	sever	sev	several	several
42	since	sinc	sint	since	since
43	still	still	stil	still	still
44	stopped	stop	stop	stopped	stop
45	student	student	stud	student	student
46	syncing	sync	synt	sync	sync
48	times	time	tim	times	time
49	tried	tri	tri	tried	try
50	unable	unabl	un	unable	unable
51	uploading	upload	upload	upload	uploading
52	use	use	us	use	use
55	wifi	wifi	wif	wifi	wifi
57	worked	work	work	worked	work
59	works	work	work	works	work
60	write	write	writ	write	write

Next steps: Generate code with `different_words` New interactive sheet

Observe: Give one over-stemming and one under-stemming example from your table.

Your answer: **Over-stemming:** The word `error` becomes `er` with LancasterStemmer. This is an example of over-stemming because the stemmer removes too much of the word, making the result difficult to recognize.

Under-stemming: The word `changed` becomes `chang` with PorterStemmer and LancasterStemmer, but `change` with WordNetLemmatizer. Stemming produces a truncated form rather than the proper dictionary form, whereas lemmatization returns the meaningful base word `change`.

Q5. Corpus statistics

Compute tokens, unique tokens and TTR for S1 (all tokens), S2 (no punctuation), S3 (final tokens) and S4 (S3 after Porter). Print the top 10 words of S1 and S3, plot log(rank) vs log(frequency) for S3, and print the top 5 bigrams of S1 and S3.

```
from collections import Counter
from nltk.stem import PorterStemmer
from nltk.util import bigrams
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

S1 = []
S2 = []
S3 = []

for text in df["Ticket_Text"]:
    processed = process_ticket(text)
    S1.extend(processed["tokens"])
    S2.extend([t for t in processed["tokens"] if t not in processed["punct"]])
    S3.extend(processed["final"])

porter = PorterStemmer()
S4 = [porter.stem(t) for t in S3]

sets = {"S1": S1, "S2": S2, "S3": S3, "S4": S4}

statistics = []

for name, tokens in sets.items():
    statistics.append({
        "Set": name,
        "Total Tokens": len(tokens),
        "Unique Tokens": len(set(tokens)),
        "TTR": len(set(tokens)) / len(tokens) if tokens else 0
    })

display(pd.DataFrame(statistics))

print("Top 10 words in S1:")
print(Counter(S1).most_common(10))

print("\nTop 10 words in S3:")
print(Counter(S3).most_common(10))

frequencies = sorted(Counter(S3).values(), reverse=True)
ranks = range(1, len(frequencies) + 1)

plt.figure(figsize=(8, 5))
plt.plot(np.log(list(ranks)), np.log(frequencies), marker="o")
plt.xlabel("Log(Rank)")
plt.ylabel("Log(Frequency)")
plt.title("Log-Rank vs Log-Frequency for S3")
plt.grid(True)
plt.show()

print("Top 5 bigrams in S1:")
print(Counter(bigrams(S1)).most_common(5))

print("\nTop 5 bigrams in S3:")
print(Counter(bigrams(S3)).most_common(5))
```

	Set	Total Tokens	Unique Tokens	TTR
0	S1	187	94	0.502674
1	S2	158	89	0.563291
2	S3	95	63	0.663158
3	S4	95	61	0.642105

```
Top 10 words in S1:
[('.', 23), ('the', 15), ('i', 7), ('yesterday', 7), ('it', 6), ("n't", 6), ('worked', 4), ('before', 3), ('assignment', 3),

Top 10 words in S3:
[('yesterday', 7), ("n't", 6), ('worked', 4), ('assignment', 3), ('deadline', 3), ('keeps', 3), ('matlab', 2), ('working', 2
```

